

DBMS COURSE PROJECT

Team 1 | Hospital Management Database

MediCare

Database Normalisation and Database Structure

Up to Third Normal Form (3NF)

Item	Details
Project	Hospital Management System
System Name	MediCare: Hospital Information and Management Database
Team	Team 1
RDBMS	MySQL / PostgreSQL / SQL Server
Scope	Normalisation of the MediCare ER-based relational design

Prepared with reference to the Team 1 project guidelines and the MediCare ER diagram.

1. Purpose of Normalisation

The purpose of normalisation in the MediCare database is to organise data into well-structured relations so that unnecessary duplication is reduced and insert, update and delete anomalies are avoided. The Team 1 project specifically requires identification of important functional dependencies and normalisation up to Third Normal Form (3NF), where applicable.

The MediCare system covers patient registration, doctors and departments, appointments, inpatient admissions and rooms, diagnoses, prescriptions and medicines, and payments. The ER design represents these concepts as separate entities/tables and uses foreign keys to connect related records.

2. Normalisation Requirements Followed

Requirement	How it is addressed
Identify functional dependencies	Functional dependencies are listed for each relation and for the original combined hospital record.
1NF	All attributes are treated as atomic/single-valued; repeating groups are separated.
2NF	Partial dependencies are removed. This is especially relevant to the composite key in prescription medicines.
3NF	Transitive dependencies are removed so non-key attributes depend on the key, the whole key, and nothing but the key.
ER-to-relational consistency	The 11 tables in the project design are retained and the PK/FK relationships are used in the normalization discussion.

3. Final MediCare Relational Structure

The following relations are based on the suggested Team 1 tables and the attributes represented in the MediCare ER diagram. Primary keys are shown as PK and foreign keys as FK.

Relation	Attributes
DEPARTMENTS	department_id (PK), name, location
DOCTORS	doctor_id (PK), name, specialisation, department_id (FK), consultation_fee
PATIENTS	patient_id (PK), name, dob, gender, phone, blood_group
APPOINTMENTS	appointment_id (PK), patient_id (FK), doctor_id (FK), appointment_date, status
ROOMS	room_id (PK), room_type, daily_charge, is_available
ADMISSIONS	admission_id (PK), patient_id (FK), room_id (FK), admit_date, discharge_date
DIAGNOSES	diagnosis_id (PK), appointment_id (FK), description, diagnosis_date
PRESCRIPTIONS	prescription_id (PK), appointment_id (FK), issued_date
MEDICINES	medicine_id (PK), name, manufacturer, unit_price
PRESCRIPTION_MEDICINES	prescription_id (PK, FK), medicine_id (PK, FK), dosage, duration_days
PAYMENTS	payment_id (PK), patient_id (FK), appointment_id (FK, nullable), admission_id (FK, nullable), amount, payment_date, mode

Note: The payment relation above reflects the ER-based design in which a payment is associated with a patient and may be linked to an appointment or an admission. If the team's implemented SQL uses a different payment structure, that implementation should be documented as a design decision.

4. Functional Dependencies

A functional dependency $X \rightarrow Y$ means that the value of X determines the value of Y . In MediCare, the primary key of each entity determines the remaining attributes of that entity.

Relation	Important functional dependency
DEPARTMENTS	department_id $\rightarrow$ name, location
DOCTORS	doctor_id $\rightarrow$ name, specialisation, department_id, consultation_fee
PATIENTS	patient_id $\rightarrow$ name, dob, gender, phone, blood_group
APPOINTMENTS	appointment_id $\rightarrow$ patient_id, doctor_id, appointment_date, status
ROOMS	room_id $\rightarrow$ room_type, daily_charge, is_available
ADMISSIONS	admission_id $\rightarrow$ patient_id, room_id, admit_date, discharge_date
DIAGNOSES	diagnosis_id $\rightarrow$ appointment_id, description, diagnosis_date
PRESCRIPTIONS	prescription_id $\rightarrow$ appointment_id, issued_date
MEDICINES	medicine_id $\rightarrow$ name, manufacturer, unit_price
PRESCRIPTION_MEDICINES	(prescription_id, medicine_id) $\rightarrow$ dosage, duration_days
PAYMENTS	payment_id $\rightarrow$ patient_id, appointment_id, admission_id, amount, payment_date, mode

4.1 Important Relationship-Based Dependencies

The foreign keys describe relationships but do not make the referenced entity's attributes depend on the foreign key inside the child table. For example, doctor_id in APPOINTMENTS identifies which doctor is involved, while the doctor's name, specialisation and department are maintained in DOCTORS. Similarly, department details are maintained in DEPARTMENTS rather than repeated for every doctor.

For PRESCRIPTION_MEDICINES, the key is composite: (prescription_id, medicine_id). The dosage and duration_days describe the particular medicine within a particular prescription, so they depend on the combination of both key attributes.

5. Unnormalised Form (UNF)

Before decomposition, a hospital-management record could be imagined as one large relation containing patient, doctor, department, appointment, admission, room, diagnosis, prescription, medicine and payment information. A simplified representation is:

HOSPITAL_RECORD(patient_id, patient_name, dob, gender, phone, blood_group, doctor_id, doctor_name, specialisation, department_id, department_name, department_location, appointment_id, appointment_date, status, diagnosis_id, description, diagnosis_date, prescription_id, issued_date, medicine_id, medicine_name, manufacturer, unit_price, dosage, duration_days, admission_id, room_id, room_type, daily_charge, admit_date, discharge_date, payment_id, amount, payment_date, mode)

This large relation would repeat department, doctor, medicine, room and patient information whenever the same entity participates in multiple transactions. Prescription-to-medicine data may also contain repeating groups because one prescription can contain multiple medicines.

6. First Normal Form (1NF)

A relation is in 1NF when every attribute contains atomic values and there are no repeating groups or multi-valued fields. The MediCare design achieves 1NF by storing one value per attribute and by separating the many-to-many prescription/medicine relationship into PRESCRIPTION_MEDICINES.

Before 1NF issue	MediCare solution
Several medicines inside one prescription record	Create one row in PRESCRIPTION_MEDICINES for each prescription-medicine pair.
Multiple appointments for one patient in one record	Store each appointment as a separate row in APPOINTMENTS.
Multiple admissions for one patient in one record	Store each admission as a separate row in ADMISSIONS.
Multiple diagnoses for appointments	Store each diagnosis as a separate row in DIAGNOSES.

1NF result: each table contains atomic attributes and each row represents one identifiable record.

7. Second Normal Form (2NF)

A relation is in 2NF when it is already in 1NF and every non-key attribute is fully dependent on the whole primary key. Partial dependency is mainly a concern when a relation has a composite primary key.

7.1 Relations with Single-Attribute Keys

DEPARTMENTS, DOCTORS, PATIENTS, APPOINTMENTS, ROOMS, ADMISSIONS, DIAGNOSES, PRESCRIPTIONS, MEDICINES and PAYMENTS use single-attribute primary keys. Therefore, a non-key attribute cannot be dependent on only part of the primary key because there is no partial key.

7.2 PRESCRIPTION_MEDICINES

PRESCRIPTION_MEDICINES has the composite primary key (prescription_id, medicine_id). Its non-key attributes are dosage and duration_days.

Relation	Key	Dependency	2NF observation
PRESCRIPTION_MEDICINES	(prescription_id, medicine_id)	(prescription_id, medicine_id) → dosage, duration_days	Dosage and duration_days describe the medicine in that specific prescription and depend on the complete composite key.

Therefore, there is no partial dependency such as prescription_id → dosage or medicine_id → dosage assumed in the design. The relation satisfies 2NF.

8. Third Normal Form (3NF)

A relation is in 3NF when it is in 2NF and no non-key attribute depends transitively on the primary key. In simple terms, non-key attributes should describe the key of their own relation and not another non-key attribute.

8.1 Example: DOCTORS and DEPARTMENTS

A poorly designed doctor relation might contain: doctor_id, doctor_name, department_id, department_name, department_location. If department_id determines department_name and department_location, then those department attributes are transitively dependent on doctor_id through department_id.

Instead, MediCare separates the information:

Relation	Dependency
DOCTORS	doctor_id → name, specialisation, department_id, consultation_fee
DEPARTMENTS	department_id → name, location

Thus, department name and location are stored once in DEPARTMENTS. DOCTORS stores only department_id as the foreign key. This removes the transitive dependency and avoids repeating department information.

8.2 Example: APPOINTMENTS and DOCTORS

APPOINTMENTS stores doctor_id as a foreign key but does not repeat doctor_name, specialisation, department_id or consultation_fee. These attributes remain in DOCTORS. Therefore, doctor information is not transitively stored inside APPOINTMENTS.

8.3 Example: ADMISSIONS and ROOMS

ADMISSIONS stores room_id as a foreign key. Room type and daily charge are maintained in ROOMS rather than being copied into every admission. This prevents room details from becoming transitively duplicated in ADMISSIONS.

8.4 Example: PRESCRIPTIONS, MEDICINES and PRESCRIPTION_MEDICINES

PRESCRIPTIONS stores prescription-specific information. MEDICINES stores medicine master information. PRESCRIPTION_MEDICINES stores the relationship-specific attributes dosage and duration_days. This separates medicine details from prescription details and from the many-to-many relationship.

8.5 Example: PAYMENTS

PAYMENTS stores payment-specific attributes such as amount, payment_date and mode. It uses foreign keys to associate a payment with the relevant patient and, where applicable, an appointment or admission. Patient, appointment and admission details are not duplicated as descriptive attributes in PAYMENTS.

9. 3NF Verification Summary

Relation	Primary Key	Reason	Normal Form
DEPARTMENTS	department_id	No partial dependency; no transitive dependency	3NF
DOCTORS	doctor_id	Department details separated into DEPARTMENTS	3NF
PATIENTS	patient_id	Patient attributes depend on patient_id	3NF
APPOINTMENTS	appointment_id	Doctor and patient details referenced by FKs, not duplicated	3NF
ROOMS	room_id	Room attributes depend on room_id	3NF
ADMISSIONS	admission_id	Room and patient referenced by FKs; admission dates depend on admission_id	3NF
DIAGNOSES	diagnosis_id	Diagnosis attributes depend on diagnosis_id	3NF
PRESCRIPTIONS	prescription_id	Prescription attributes depend on prescription_id	3NF
MEDICINES	medicine_id	Medicine master attributes depend on medicine_id	3NF
PRESCRIPTION_MEDICINES	(prescription_id, medicine_id)	Non-key attributes depend on the full composite key	3NF
PAYMENTS	payment_id	Payment attributes depend on payment_id; related records referenced by FKs	3NF

10. Anomalies Avoided

Anomaly	How the MediCare design reduces it
Update anomaly	Department, doctor, room and medicine master details are stored in their own relations, so one change does not require changing many repeated transaction rows.
Insert anomaly	A department, doctor, medicine or room can be created independently rather than requiring an unrelated appointment or payment record.
Delete anomaly	Deleting an appointment does not require deleting the doctor, patient, medicine or department master information.

11. Normalisation Outcome

The final MediCare design is organised into 11 relations: DEPARTMENTS, DOCTORS, PATIENTS, APPOINTMENTS, ROOMS, ADMISSIONS, DIAGNOSES, PRESCRIPTIONS, MEDICINES, PRESCRIPTION_MEDICINES and PAYMENTS. The decomposition separates master data from transaction/relationship data and uses primary and foreign keys to maintain relationships.

The design satisfies the project requirement to identify functional dependencies and demonstrate normalization up to 3NF. The most important normalization case is PRESCRIPTION_MEDICINES, where the composite key ensures that dosage and duration_days depend on the complete prescription-medicine combination.

12. Design Rationale and Assumptions

- The project follows the Team 1 requirement to normalise the database up to 3NF where applicable.
- The M:N relationship between prescriptions and medicines is represented using the PRESCRIPTION_MEDICINES bridge table.
- A department can have many doctors, while each doctor belongs to one department.
- A patient can have many appointments and admissions over time.
- A room can be used by multiple admissions over time, while the project assumes only one active admission per room at a time.
- Payment records are kept separate from appointment/admission details to avoid repeating transaction and master data.
- If the implemented SQL schema differs from an ER attribute or relationship, the final project report should document the change and its justification, as required by the Team 1 project guidelines.